

Project 1: Elevator System Specification and Refinement

MFS Course 2025/26

Eduardo Machado - up202105337

Lino - [ID]

Giovanni - [ID]

April 2026

1 Part 1: Baseline Validation and Model Understanding

1.1 Baseline Configuration and Liveness Overhead

To establish a baseline, we validated `Elevator.tla` using a minimal instance. We specifically isolated the `TemporalInvariant` to measure its impact on the model checker's performance.

Model Parameters:

- **People:** $\{p_1, p_2\}$, **Elevators:** $\{e_1\}$, **Floors:** 3

Metric	Safety Only	Full (Liveness)
Distinct States	2,685	2,685
Diameter	14	14
Execution Time	3s	11s
Result	Passed	Passed

Table 1: Baseline results demonstrating the $\approx 266\%$ time overhead for cycle detection in liveness checking.

1.2 State Space Explosion and Sensitivity Analysis

We explored how the state space grows when scaling different parameters. We observed that increasing the number of people (`Congestion`) impacts the state space more severely than increasing the number of elevators (`TwoElevators`).

Model	Config (E, P, F)	Distinct States	Time (Safety)	Liveness Status
Baseline	1E, 2P, 3F	2,685	3s	Verified (11s)
TwoElevators	2E, 2P, 3F	50,653	10s	Timeout (12m)
Congestion	1E, 4P, 3F	906,693	31s	Prohibitive

Table 2: Scaling sensitivity results.

1.2.1 Visualizing Growth

The following trend demonstrates that the complexity of the system is not linear. Adding a second elevator increased the state space by $\approx 18x$, while doubling the number of people increased it by over $\approx 330x$.

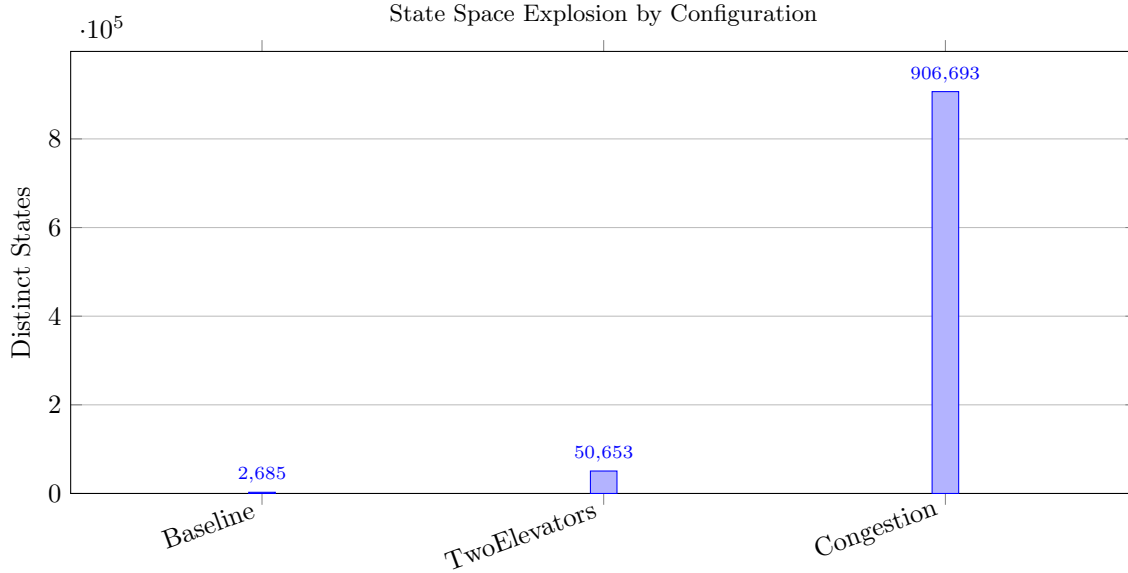


Figure 1: Distinct states by configuration.

1.3 Documentation of expensive properties

As evidenced by the **TwoElevators** run, the **TemporalInvariant** is significantly more "expensive" than safety invariants. While TLC explored all 50,653 states and verified safety in 10 seconds, it failed to complete the liveness check (cycle detection) within 12 minutes. This is because cycle detection requires searching for loops in the behavior graph, which has a higher computational complexity than the state-by-state validation used for safety.

1.4 Operational Role of Actions

The operational logic of the elevator system is governed by the following key actions:

- **PickNewDestination(p)**: Represents a person's decision to travel. It updates a person's **destination** without immediately requesting the elevator.
- **CallElevator(p)**: The person requests service from their current floor, adding a request to **ActiveElevatorCalls**.
- **DispatchElevator(c)**: The system's controller assigns an elevator to a call based on proximity and current direction.
- **MoveElevator(e)**: Moves the elevator car between floors, provided the doors are closed and a direction is set.
- **OpenElevatorDoors(e)**: Occurs when an elevator arrives at a floor with a pending call or an internal button press.
- **EnterElevator(e)**: Facilitates the boarding process for people waiting for the elevator's current direction.
- **ExitElevator(e)**: People whose destination matches the current floor leave the car, resetting their **waiting** status to **FALSE**.
- **CloseElevatorDoors(e)**: Ensures the elevator cannot move until all eligible boarding/exiting actions are completed.
- **StopElevator(e)**: Resets an elevator to a **Stationary** state once all its assigned calls are fulfilled.

2 Part 2: Fairness and Liveness Sensitivity Study

2.1 Controlled Fairness Experiment

To analyze the impact of fairness assumptions on system progress, we conducted a sensitivity study by weakening the fairness requirements for the `DispatchElevator` action. Specifically, the **Strong Fairness** (SF) assumption on the dispatcher was replaced with **Weak Fairness** (WF) in the `TemporalAssumptions` definition:

- **Original:** $\forall c \in \text{ElevatorCall} : \text{SF_Vars}(\text{DispatchElevator}(c))$
- **Modified:** $\forall c \in \text{ElevatorCall} : \text{WF_Vars}(\text{DispatchElevator}(c))$

2.2 Results and Liveness Violation

Testing this modification on the baseline model (1 Elevator, 2 People, 3 Floors) resulted in a **Temporal Property Violation**. TLC identified a starvation scenario where the `TemporalInvariant` failed to hold.

Fairness Assumption	Liveness Result
Strong Fairness (SF)	Passed (Verified in 11s)
Weak Fairness (WF)	Failed (Counterexample found in 7s)

Table 3: Comparison of fairness impacts on liveness verification.

2.3 Counterexample Analysis: Passenger Starvation

The error trace provided by TLC revealed a "livelock" or starvation cycle. In this scenario, passenger p_1 at Floor 1 issues an "Up" call to Floor 2. However, p_1 is never serviced because the system enters an infinite loop where the elevator prioritizes passenger p_2 .

Mechanism of Failure:

1. p_1 sets `waiting = TRUE`, enabling the `DispatchElevator` action for that floor.
2. Passenger p_2 repeatedly requests rides between Floors 2 and 3.
3. As the elevator moves to service p_2 or opens its doors, the conditions that make the elevator "available" to be dispatched for p_1 (e.g., being stationary or moving in the correct direction) are momentarily interrupted.
4. Under **Weak Fairness**, the system is only required to execute an action if it remains **ENABLED** *continuously*. Because the enablement of the dispatcher for p_1 "flickers" as the elevator services p_2 , the system is mathematically permitted to ignore p_1 indefinitely.

2.4 Conclusion on Fairness Sensitivity

This experiment demonstrates that **Strong Fairness** is a strict requirement for the `DispatchElevator` action. Unlike **Weak Fairness**, **Strong Fairness** guarantees that if an action is enabled *infinitely often* (even if interrupted), it must eventually occur. In a multi-passenger environment where concurrent requests compete for resources, SF ensures that every call is eventually dispatched, regardless of how "busy" the elevator is with other passengers.

3 Part 3: Custom Properties and Verification

In accordance with the project requirements for a three-member group, we implemented and verified three additional properties. These properties cover individual elevator safety, multi-elevator coordination, and passenger progress.

3.1 Property 1: Physical Motion Safety

Definition: This invariant ensures an elevator car does not transition floors while doors are open. $\text{DoorsClosedWhileMoving} := \forall e \in \text{Elevator} : \text{ElevatorState}[e].\text{direction} \neq \text{"Stationary"} \Rightarrow \neg \text{ElevatorState}[e].\text{doorsOpen}$

- **Status:** Failed.
- **Analysis:** TLC identified a violation during the `DispatchElevator` action. In this specification, the `direction` variable represents the *intent* or assignment of the car. Since the car is assigned a direction before the doors finish closing at its current floor, the property is violated.
- **Insight:** This failure highlights the distinction between high-level logic (intent) and physical system states (motion) within a TLA+ specification.

3.2 Property 2: Multi-Elevator Coordination

Definition: This invariant ensures that two different elevators never open their doors on the same floor simultaneously, preventing redundant service and passenger confusion. $\text{NoDualDoorOpen} := \forall e_1, e_2 \in \text{Elevator} : (e_1 \neq e_2 \wedge \text{ElevatorState}[e_1].\text{doorsOpen} \wedge \text{ElevatorState}[e_2].\text{doorsOpen}) \Rightarrow \text{ElevatorState}[e_1].\text{floor} \neq \text{ElevatorState}[e_2].\text{floor}$

- **Status:** Passed.
- **Configuration:** Verified on `model.TwoElevators` (50,653 states).
- **Analysis:** The success of this property confirms that the dispatcher logic effectively partitions the floor space, ensuring that once a call is assigned, other cars do not interfere with that specific floor request.

3.3 Property 3: Guaranteed Boarding (Liveness)

Definition: This temporal property ensures that a passenger in a `waiting` state will eventually enter an elevator car, verifying the absence of "platform starvation." $\text{EventuallyPickedUp} := \forall p \in \text{Person} : \text{PersonState}[p].\text{waiting} \leadsto (\text{PersonState}[p].\text{location} \in \text{Elevator})$

- **Status:** Passed.
- **Configuration:** Verified on `model.baseline` (2,685 states).
- **Analysis:** Unlike the safety invariants, this property is a "Progress" guarantee. It relies entirely on the **Strong Fairness** assumptions established in Part 2. Its success confirms that our fairness constraints are sufficient to ensure that the dispatcher cannot ignore a call indefinitely, even if other concurrent requests are available.

3.4 Comparative Summary of Properties

The following table summarizes the differences in verification complexity and results across the three custom properties.

Property	Type	Status	Fairness Sensitivity
<code>DoorsClosedWhileMoving</code>	Safety Invariant	Failed	Low (Structural)
<code>NoDualDoorOpen</code>	Safety Invariant	Passed	Low (Structural)
<code>EventuallyPickedUp</code>	Liveness Property	Passed	High (Required SF)

Table 4: Comparison of custom properties and verification results.